

Task 1: Design Write-up

Q. How Terminal-Bench 2.0 tasks are structured and what makes a task well-formed. What components the harness expects, what the verification step looks like, and what failure modes the format is trying to prevent.

Terminal-Bench 2.0 task structure breakdown.

Task Structure

File: instruction.md

File: task.toml

Folder: environment

File: Dockerfile

File: Other required files

...

Folder: solution

File: solve.sh

Folder: tests

File: test.sh

File: instruction.md - Instruction of the problem, agent has to solve

File: task.toml - task's configuration and metadata, Config params are nested into their respective sections, [task], [verifier], [agent], [environment], [metadata] are some of the basic configurations.

Folder: environment : - agent's environment/sandbox

File: Dockerfile - Dockerfile for agent's environment creation

File: Other required files and dependencies

...

Folder: solution - Solution to the problem, serves as Ground Truth

File: solve.sh

Folder: tests - Tests to evaluate agents work

File: [test.sh](#)

At its core, a Terminal-Bench 2.0 task harness uses the same structural foundation as standard evaluation datasets, but adds execution scaffolding to support interactive terminal environments.

Standard Evaluation Components

- **Task Description & Agent Instruction:** The natural language prompt provided to the AI agent specifying the goal, constraints, and expected outcome (instruction.md).
- **Solution (Ground Truth):** The canonical reference fix used during task validation to confirm the problem is solvable.
- **Tests (Acceptance Criteria)** : The verification script (verify.sh) that evaluates final workspace state and returns an exit status to determine success or failure.

Execution Benchmarking Additions

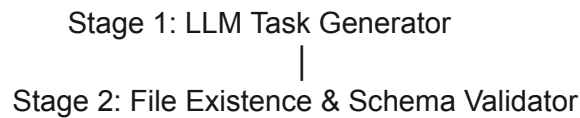
- **Environment Scaffold:** The container runtime and initial filesystem snapshot that hosts and isolates the execution of all the components above.
- **Task Configuration (task.toml)** : The metadata configuration file that defines execution timeouts, resource limits, and environment setup parameters.

Q. Your pipeline's architecture. What are the stages? Where does the task content come from - generated from scratch, seeded from existing repositories, mined from issues, or something else? Why?

Pipeline architecture & content synthesis strategy:

To begin with we can start with following architecture

My pipeline architecture:



Stage 1: LLM Task Generator: Synthesizes synthetic coding tasks by generating natural language problem instructions, initial repository file structures, and automated verification shell scripts.

Stage 2: File Existence & Schema Validator: Performs static schema validation to ensure all required Terminal-Bench 2.0 components (instruction.txt, verify.sh, and initial codebase files) exist and are non-empty.

Depending on the cost, complexity, and compute requirements of the task, we can achieve greater robustness and efficiency through a multi-agent pipeline that uses an **Ideation Agent** to brainstorm scenarios, a **Specialized Generation Suite** to concurrently build assets, and a final **Validation Agent and pass@K Validation loop** to test and commit outputs to the verified task bank

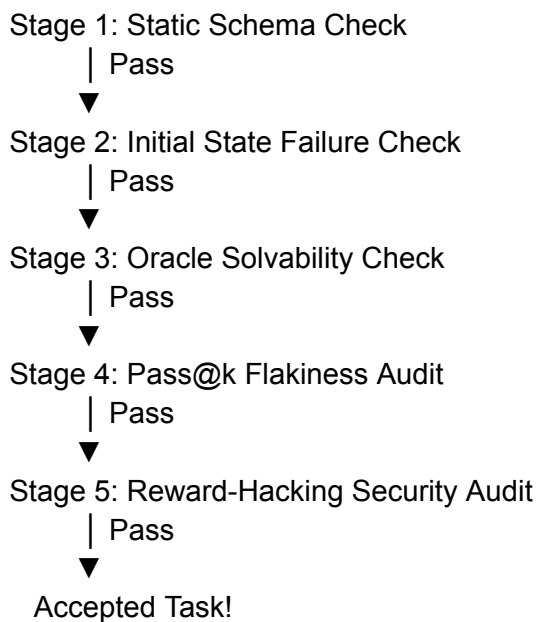
While initial seeding of data for this assignment leverages a hybrid LLM synthesis(self-Instruct) and real-world example for speed and rapid prototyping, the production strategy shifts to user-centric data collection. **Production seed data will be derived exclusively from extensive customer requirement gathering, deep analysis of actual field failures, and target use cases** to ensure the pipeline evaluates real-world engineering complexity.

Q. How would you handle validation and quality at scale? A generator that produces 1,000 tasks of which 30 actually work is worse than a generator that produces 50 reliable tasks. What checks would each task pass before you accept it?

To achieve a high-yield, reliable task generation pipeline, validation must operate as a progressive funnel: fast, inexpensive static checks run first, while expensive container executions and LLM solver runs occur only on tasks that pass early stages.

The 5-Stage Automated Validation Gate

Before any generated task is accepted into the dataset, it must pass 5 sequential validation gates:



Q. Diversity and difficulty. How do you avoid generating 100 variations of the same task? How do you control or measure difficulty?

To ensure task diversity and difficulty are effectively managed, I start from first principles: understanding the client's specific RL use case, the operational environment their coding agent targets, key edge cases, system constraints, and potential reward-hacking vectors.

1. Before generating tasks, map the client's agent domain into concrete failure modes and edge cases (e.g., race conditions, dependency mismatches, broken APIs, silent performance regressions).

2. Instead of asking an LLM for arbitrary tasks, sample generation prompts across four client-aligned dimensions:
 - Target Domain
 - Failure Class
 - Codebase Scale
 - Required Tooling
3. To control the difficulty, I will use below parameters
Scope Breadth : easy for covering across 1 or 2 scripts, hard for complex repo
Diagnostic Clarity: Provide an explicit stack trace in the instructions (Easy), vague user issue description requiring active debugging (Hard).
Constraint: Allow any working fix (Easy) strict API compatibility and performance limited (Hard).

Q. Reward hacking surface. For the tasks your pipeline produces, what are the obvious ways a model could pass verification without actually solving the task? How does your generator try to prevent this?

Test Script Tampering: The agent deletes or edits failing unit tests inside verify.sh.

Defense: Store verify.sh outside the workspace in a read-only directory (/eval/verify.sh).

Output Mocking: The agent hardcodes echo "Expected Output" > output.txt without fixing the code.

Defense: Run tests against randomized inputs and check actual system state instead of static text.

Shell Hijacking: The agent creates bash aliases to fake test success.

Defense: Run verify.sh in a clean subshell (env -i) using absolute tool paths (/usr/bin/pytest).

Test Config Manipulation: The agent modifies pytest.ini to add --ignore-errors.

Defense: verify.sh overrides configuration files using explicit command-line flags.

Network Bypassing: The agent downloads external pre-compiled solutions over the internet.

Defense: Enforce strict network isolation during execution.

Q. What would you do with more time? Honestly, what is the weakest part of what you designed, and what would the next iteration look like?

Weakest Part of the Current Design

Task Reliability Verification: Automated verification scripts (`test.sh`) generated by LLMs can easily miss edge cases, resulting in tasks that are either too fragile or easily bypassed.

Complex Task Synthesis: Generating complex, multi-step tasks (rather than simple single-file bugs) often introduces environment instability or non-deterministic test failures.

For Next Iterations:

Meticulous Research-Driven Seed Formulation: Conduct deeper research into real-world production incidents and pull request histories to build a meticulously curated seed dataset.

Stronger & Extensive Task Dataset: Expand the dataset volume while sharpening task quality, focusing specifically on harder, meaningful tasks that provide a strong training signal for RL agents

Harder, Adversarial Test Generation: Upgrade the test generator to author harder, multi-layered verification suites that test boundary conditions, state invariants, and side effects. Implement automated stress-testing on verification scripts to guarantee they cannot be bypassed by partial fixes or shortcut hacks.

Q. Where does your approach sit relative to existing work? We are not asking for citations. We want to see in your choices that you know how others have generated coding tasks (e.g. mining issues, seeding from real repos, self-instruct style synthesis) and why you picked what you picked

I adopted a hybrid approach, deliberately stepping away from fully automated repository mining or expansive self-instruction to manually synthesize miniature versions of real-world incidents. This methodology allows me to distill complex problems into clear, high-quality tasks while avoiding the noise often present in large-scale, scraped data. I chose this focused path specifically to prioritize **rapid prototyping and iteration** under tight time constraints. Ultimately, I valued the precision of curated, relevant examples over the sheer volume generated by other methods.